GrokRxiv:2026.04.c2rust-compile-time-supremacy [cs.SE] 08 Apr 2026

Compile-Time Supremacy:

A Strategic Architecture for AI-Assisted C→Rust Migration at Scale

Matthew Long

The YonedaAI Collaboration

YonedaAI Research Collective · Magneton Labs LLC

Chicago, IL

matthew@yonedaai.com · https://yonedaai.com

April 8, 2026

Abstract

The convergence of three forces—DARPA's TRACTOR program, Microsoft's stated goal to eliminate all C/C++ by 2030, and the demonstrated capacity of large language models to generate and transform code—creates a once-in-a-generation commercial opportunity in automated C→Rust translation. However, general-purpose LLMs are structurally insufficient for production-grade memory-safety migration. This paper presents a strategic architecture for building a specialized AI-assisted C→Rust translation toolchain, grounded in the *Compile-Time Supremacy* thesis: that the migration from C to Rust is fundamentally a problem of *inferring missing type information* that C never required developers to state. We decompose the system into five pipeline stages, define the open-source and proprietary boundaries, specify the training data strategy including synthetic verified-pair generation, and outline a practical build path using Claude as the foundational reasoning engine augmented by specialized static analysis, formal verification, and domain-specific fine-tuned models. The architecture positions a new venture—**Ferrous Bridge**—to capture the emerging $10B+ market in legacy code migration services.

1 The Thesis: Compile-Time Supremacy as Business Strategy

1.1 The Minimal Runtime Axiom Applied to Language Migration

The Minimal Runtime Axiom states: *every invariant deferred to runtime is an invariant that will eventually be violated.* C is the canonical embodiment of maximal runtime deferral—raw pointer arithmetic, manual memory management, implicit type coercions, and undefined behavior all represent invariants that exist only in the programmer's mental model, never in the compiler's enforcement regime.

Rust inverts this relationship. The borrow checker, lifetime system, and ownership model move invariants from "trust the programmer" to "prove it at compile time." The entire C→Rust migration is therefore not a syntactic translation problem but a *type inference* problem: recovering the implicit ownership semantics, lifetime relationships, and aliasing constraints that C code encodes only in programmer intent.

Principle 1.1 (Compile-Time Supremacy)**.** The economic value of C→Rust migration is proportional to the quantity of implicit runtime invariants successfully promoted to compile-time enforcement. A translation that produces `unsafe` Rust wrapping C semantics creates zero security value. Only translation that **infers and encodes** the missing type information produces genuine memory safety.

1.2 Why General-Purpose LLMs Fail

General-purpose LLMs can perform surface-level C→Rust translation with surprising quality on isolated functions. They fail systematically on the hard cases that constitute the actual security value:

1. **Cross-function lifetime inference.** The lifetime of a pointer returned from function f and consumed by function g requires whole-program dataflow analysis. LLMs have a fixed context window and no persistent symbolic state.

2. **Pointer aliasing.** C permits arbitrary aliasing; Rust's borrow checker enforces exclusivity. Determining which aliases are "accidental" (and can be refactored) versus "essential" (and require `RefCell` or `Arc`) requires semantic understanding of programmer intent.

3. **Undefined behavior exploitation.** Compilers *optimize based on UB assumptions.* Translated Rust code that naively preserves C behavior may preserve behavior that was never intended—the C compiler was silently exploiting UB for optimization.

4. **Custom allocators and arena patterns.** Large C codebases implement bespoke memory management (arena allocators, slab allocators, pool allocators) that must be mapped to Rust's ownership model, not transliterated.

5. **Verification of semantic equivalence.** The translated Rust must do what the C *was supposed to do*, not what it *actually did* (since what it actually did may include exploitable bugs). This requires a specification oracle that no LLM possesses natively.

The conclusion: a specialized system must combine LLM reasoning with formal static analysis, dynamic testing, and human-in-the-loop verification. No single component suffices.

2 Market Analysis: The C→Rust Migration Opportunity

2.1 Demand Signals

Three institutional demand signals converge simultaneously:

1. **DARPA TRACTOR** (2024–ongoing). $5M+ awards to academic teams (Illinois/Wisconsin/Berkel MIT Lincoln Lab performing T&E. Explicit goal: automate C→Rust at production quality. This validates the problem and creates a government procurement channel.

2. **Microsoft's 2030 commitment.** Distinguished Engineer Galen Hunt's stated goal to eliminate all C/C++ from Microsoft by 2030. Internal team building AI agents targeting "1 engineer, 1 month, 1 million lines." This validates enterprise demand at the largest possible scale.

3. **White House ONCD directive.** The Office of the National Cyber Director's 2024 report calling for universal adoption of memory-safe languages. This creates regulatory pressure across the defense industrial base, critical infrastructure, and government contractors.

2.2 Market Sizing

Table 1: C→Rust Migration Market Segments

Segment	**Description**	**Lines (est.)**	**TAM**
Defense/IC	DoD, intelligence community legacy systems	5B+	$3–5B
Enterprise OS	Windows, Linux kernel, Android native	2B+	$2–4B
Embedded/IoT	Automotive, medical devices, industrial control	10B+	$3–6B
Financial	Trading systems, banking infrastructure	1B+	$1–2B
Telecom	Network infrastructure, 5G stack	2B+	$1–2B
Open Source	OpenSSL, curl, nginx, etc.	500M+	$200M–500M

Total addressable market: $10–20B over a 10-year migration cycle, with the critical constraint being that **no adequate tooling exists today**. The team that builds the first production-grade automated translator captures outsized first-mover advantage.

2.3 Competitive Landscape

Table 2: Current C→Rust Players

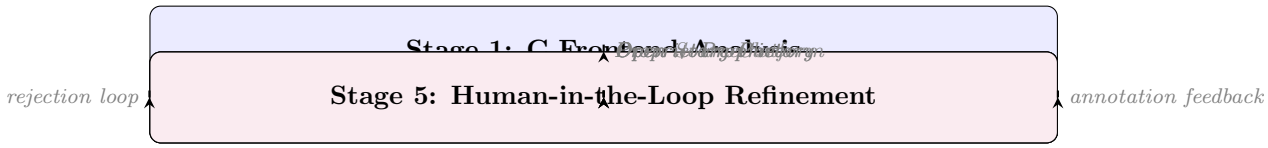
Player	Approach	Gap
Immunant (C2Rust)	AST-level transpilation, DARPA-funded	Produces `unsafe` Rust
ForCLift (Berkeley)	Verified lifting, formal methods + LLM	Research stage
Microsoft (internal)	AI agents at scale	Internal only
Code Metal	Transpilation for edge hardware	Hardware focus
General LLMs	Prompt-based translation	No verification

The critical gap: **no current player combines LLM-powered semantic reasoning with formal verification and produces safe, idiomatic Rust.** Immunant's C2Rust produces mechanically correct but `unsafe` Rust. Academic approaches remain in research. Microsoft's tools are internal. The market is wide open for a productized solution.

3 System Architecture: The Ferrous Bridge Pipeline

We propose a five-stage pipeline architecture named **Ferrous Bridge** (iron → iron oxide → the bridge from C to Rust). Each stage has distinct open-source and proprietary components.

3.1 Architecture Overview



3.2 Stage 1: C Frontend Analysis (Open Source)

Purpose: Parse C source into a rich intermediate representation that captures everything an LLM needs to reason about ownership and lifetimes.

Components:

- **Clang-based AST extraction** with full type information, macro expansion, and preprocessor resolution
- **LLVM IR emission** for optimization-level semantic analysis
- **Points-to analysis** (Andersen-style or Steensgaard) to build an initial aliasing graph
- **Call graph construction** with indirect call resolution via type-based devirtualization

- **Use-def chain extraction** for every pointer, with cross-function tracking
- **Dynamic analysis harness** using sanitizers (ASan, MSan, TSan) and fuzzing (libFuzzer, AFL++) to observe runtime behavior

Open-source rationale: This stage uses well-established compiler infrastructure. Open-sourcing it builds community trust, attracts contributors familiar with LLVM/Clang, and creates a funnel to the proprietary stages. It also satisfies DARPA's publication requirements for TRACTOR-adjacent work.

Output: A `C-Analysis Bundle` (CAB)—a serialized JSON/protobuf package containing the AST, CFG, call graph, points-to graph, use-def chains, and dynamic analysis traces.

3.3 Stage 2: Semantic Graph Construction (Open Core + Proprietary)

Purpose: Transform the CAB into an *Ownership Semantic Graph* (OSG)—a typed intermediate representation that makes implicit C ownership patterns explicit.

Key innovations:

- **Ownership inference engine.** For each pointer in the program, classify it as: *owning* (will be freed by this scope), *borrowing* (read-only reference to owned data), *mutably borrowing* (exclusive mutable reference), or *shared* (requires `Arc`/`Rc`). This is the core type-inference problem.
- **Lifetime region analysis.** Construct candidate Rust lifetime regions from use-def chains and dominator trees. Each region represents a borrow scope.
- **Allocator pattern recognition.** Detect arena, pool, slab, and custom allocator patterns and map them to idiomatic Rust equivalents (`bumpalo`, `typed-arena`, custom allocator API).
- **Idiom detection.** Recognize C patterns that map to Rust standard library idioms: linked lists → `Vec`, callback+context → closures, error-code returns → `Result<T, E>`.

Open/proprietary split:

- *Open:* Graph data structures, serialization format, basic ownership heuristics
- *Proprietary:* Trained ownership classifier, idiom detection models, allocator pattern library

3.4 Stage 3: LLM-Powered Translation Engine (Proprietary Core)

Purpose: Given the OSG, generate safe, idiomatic Rust code at the function and module level.

This is the proprietary heart of the system. The key insight is that the LLM is *not* doing raw C→Rust translation. It receives the C source *annotated with the OSG's ownership/lifetime/idiom analysis*. The LLM's job is to synthesize Rust code that satisfies the inferred type constraints while preserving semantic equivalence.

Model architecture:

- **Base model:** Claude (via API) as the primary reasoning engine for complex translation decisions, architectural pattern recognition, and natural language explanation of translation choices.
- **Specialized fine-tuned model:** A smaller model (8–70B parameters) fine-tuned on verified C↔Rust translation pairs, optimized for high-throughput batch translation of routine code patterns. Candidate base: CodeLlama, StarCoder2, DeepSeek-Coder, or Qwen2.5-Coder.
- **Routing logic:** A classifier determines whether each translation unit should be routed to the fast fine-tuned model (routine patterns: simple functions, data structure translations, standard API mappings) or to Claude (complex patterns: lifetime inference across module boundaries, custom allocator migration, concurrent code transformation).

Training data strategy for the fine-tuned model:

1. **Curated corpus.** Mine GitHub for projects that have been manually rewritten from C to Rust (e.g., `rustls` vs. OpenSSL, `sudo-rs` vs. sudo, `ntpd-rs` vs. ntpd). Extract function-level aligned pairs.
2. **Synthetic generation via Claude.** Use Claude to translate C functions to Rust, then verify with `rustc` (type-checks), Miri (UB detection), and property-based tests. Verified pairs enter the training corpus.
3. **C2Rust + manual refinement.** Run Immunant's C2Rust to produce `unsafe` Rust, then use Claude to "safe-ify" the output. Verify. The (C, unsafe-Rust, safe-Rust) triples are extremely high-value training data.
4. **Adversarial generation.** Deliberately create C code with known memory safety bugs. Train the model to produce Rust code that *does not reproduce the bug*—forcing it to infer programmer intent rather than transliterating behavior.

3.5 Stage 4: Formal Verification & Equivalence Checking (Open Core + Proprietary)

Purpose: Prove that the generated Rust is semantically equivalent to the C source (modulo intentionally fixed bugs) and is genuinely memory-safe.

Components:

- **Rust compiler as oracle.** If `rustc` accepts the code without `unsafe` blocks, it is memory-safe by construction. This is the minimum bar.

- **Miri execution.** Run the Rust code under Miri (Rust's UB detector) to catch subtle issues the type system alone cannot.
- **Kani model checking.** Use Amazon's Kani to formally verify critical functions against specifications extracted from the C code's test suite and documentation.
- **Differential fuzzing.** Compile both the C and Rust versions, fuzz both with identical inputs, compare outputs. Divergence triggers a rejection loop back to Stage 3.
- **Creusot/Prusti verification.** For high-assurance targets (defense, medical, automotive), use Creusot or Prusti to generate formal proofs of functional correctness.

Open/proprietary split:

- *Open:* Differential fuzzing harness, Miri integration, basic test generation
- *Proprietary:* Specification extraction from C code, Kani proof orchestration, verification-guided re-translation logic

3.6 Stage 5: Human-in-the-Loop Refinement (Proprietary Platform)

Purpose: A web-based IDE/dashboard where human Rust experts review, annotate, and approve translations. Every human decision feeds back into the training pipeline.

Features:

- Side-by-side C/Rust diff view with OSG annotations
- Confidence scoring per translation unit (green/yellow/red)
- One-click approval that adds the verified pair to the training corpus
- Annotation tools for correcting ownership classifications, which retrain the Stage 2 classifier
- Audit trail for compliance (defense, automotive, medical certifications)

4 Open Source vs. Proprietary Boundary

The open/closed boundary is the strategic crux. The principle: **open-source the infrastructure that creates ecosystem lock-in; keep proprietary the intelligence that creates competitive advantage**.

Table 3: Open/Proprietary Component Map

Component	Description	Open	Closed
Clang/LLVM frontend	AST/IR extraction, points-to analysis	✓	
CAB format spec	Serialization format for analysis bundles	✓	
OSG data structures	Ownership graph representation	✓	
Ownership classifier	ML model for pointer classification		✓
Idiom detection engine	Pattern matching for C→Rust idioms		✓
Fine-tuned code model	Specialized C→Rust translation LLM		✓
Claude integration layer	Routing, prompt engineering, orchestration		✓
Differential fuzzing harness	Equivalence testing framework	✓	
Miri/Kani integration	Verification pipeline connectors	✓	
Specification extractor	Infer specs from tests and docs		✓
Verification orchestrator	Proof strategy selection and execution		✓
Training data pipeline	Synthetic pair generation and curation		✓
Human review platform	Web IDE for expert review		✓

4.1 Strategic Rationale

Why open-source the frontend: The C analysis infrastructure benefits from community contribution. Every contributor who builds on our CAB format is a potential customer for the proprietary translation engine. The format becomes a standard, creating switching costs. DARPA and government buyers strongly prefer open-source infrastructure components.

Why keep the models proprietary: The fine-tuned translation model and ownership classifier are the competitive moat. Training data (verified C↔Rust pairs) is expensive to generate and curate. The Claude integration layer embodies accumulated prompt engineering and routing intelligence that represents months of iteration.

Why open-source the verification harness: Trust. Defense and enterprise buyers will not trust a black-box translator. Open-sourcing the verification pipeline lets them audit the correctness guarantees independently while still requiring the proprietary engine to generate the code being verified.

5 Building with Claude: The Practical Path

5.1 Claude's Role in the Architecture

Claude serves three distinct functions in the Ferrous Bridge pipeline:

1. **Complex translation oracle.** For translation units that exceed the fine-tuned model's confidence threshold—typically those involving cross-module lifetime inference, custom allocator migration, or concurrent code—Claude receives the C source

annotated with the OSG and generates candidate Rust translations. Claude's strength is reasoning about *programmer intent* from context, comments, variable names, and architectural patterns.

2. **Synthetic training data generator.** Claude generates candidate C→Rust translations at scale. Each candidate is verified by the Stage 4 pipeline. Verified pairs are added to the fine-tuned model's training corpus. This creates a *distillation flywheel*: Claude's reasoning ability is progressively distilled into the smaller, faster, cheaper fine-tuned model.

3. **Explanation engine.** For the human-in-the-loop stage, Claude generates natural language explanations of *why* each translation decision was made. This is critical for defense and regulated industry buyers who require audit trails and justifications.

5.2 Claude Code as Development Accelerator

The development of Ferrous Bridge itself should be built using Claude Code with agent teams:

- **Agent Team 1: Frontend.** Builds the Clang-based analysis pipeline. Claude Code agents write the AST extraction, points-to analysis, and CAB serialization code.
- **Agent Team 2: OSG Engine.** Implements the ownership semantic graph construction. Agents write the ownership classifier, lifetime region analysis, and idiom detection.
- **Agent Team 3: Translation.** Builds the Claude API integration, prompt templates, routing logic, and fine-tuning data pipeline.
- **Agent Team 4: Verification.** Implements the differential fuzzing, Miri/Kani integration, and verification orchestration.
- **RalphD as meta-supervisor.** Monitors drift across all four agent teams, ensuring architectural consistency, enforcing the OSG schema contract, and detecting when agents diverge from the design spec.

5.3 Phase 1: MVP (Months 1–3)

1. Stand up the Clang frontend. Parse a curated set of 50 small-to-medium C projects (<10K LOC each). Emit CABs.
2. Build a minimal OSG with manual ownership annotations. No ML yet—use heuristics (e.g., "if `malloc` and `free` are in the same function, it's owning").
3. Integrate Claude API. For each function in the CAB, send C source + OSG annotations to Claude with a carefully engineered prompt. Collect Rust output.
4. Run `rustc` on every output. Measure: (a) compilation success rate, (b) percentage of `unsafe` blocks, (c) differential fuzzing pass rate.

5. **Target:** 60% compilation success, <20% unsafe blocks on simple projects.

5.4 Phase 2: Training Data Flywheel (Months 4–8)

1. Use Phase 1 results to build the initial training corpus. Every Claude-generated translation that passes verification becomes a training pair.
2. Fine-tune a code model (starting with DeepSeek-Coder-V2 or Qwen2.5-Coder-32B) on the verified pairs.
3. Build the routing classifier: simple functions go to the fine-tuned model, complex functions go to Claude.
4. Scale to 500 projects. Build the ownership classifier using supervised learning on the manually annotated OSGs from Phase 1.
5. **Target:** 80% compilation success, fine-tuned model handles 70% of functions, Claude handles 30%.

5.5 Phase 3: Verification and Productization (Months 9–14)

1. Integrate Kani for formal verification of critical functions.
2. Build the human review platform (web IDE).
3. Launch beta with 3–5 design partners (defense contractors, embedded systems companies).
4. Apply to DARPA TRACTOR as a performer or subcontractor.
5. Open-source the frontend and verification harness.
6. **Target:** 90%+ compilation success, <5% unsafe blocks, formal verification on critical paths.

5.6 Phase 4: Scale and Defensibility (Months 15–24)

1. Continuous fine-tuning from human review feedback (every approved translation improves the model).
2. Expand to C++ (significantly harder—templates, RAII overlap, virtual dispatch).
3. Build domain-specific modules: automotive (MISRA C compliance), medical (IEC 62304), defense (DO-178C).
4. Pursue SBIR/STTR funding for defense-specific applications.
5. **Target:** 95%+ success on C, initial C++ support, 3+ paying enterprise customers.

6 Training Data Strategy: The Verified Pair Problem

The fundamental bottleneck is training data. There are approximately 100x more C code samples than Rust in public repositories, and aligned C↔Rust translation pairs are extremely scarce. The strategy is synthetic generation with formal verification as the quality gate.

6.1 Data Sources

1. **Natural rewrites.** Projects that have been manually rewritten from C to Rust: `rustls`/OpenSSL, `sudo-rs`/sudo, `ntpd-rs`/ntpd, `hickory-dns`/BIND. Function-level alignment yields approximately 5,000–10,000 high-quality pairs.

2. **C2Rust + safe-ification.** Run C2Rust on open-source C projects. The output is mechanically correct `unsafe` Rust. Use Claude to generate safe Rust equivalents. Verify with `rustc` + Miri + differential fuzzing. This can generate 50,000–100,000 pairs at moderate cost.

3. **Pure synthetic generation.** Generate C functions from templates covering known difficulty patterns (linked lists, trees, hash maps, arena allocators, callback patterns, signal handlers). Translate with Claude. Verify. 100,000+ pairs possible.

4. **Adversarial pairs.** C code containing *known bugs* (CVEs from NVD, synthetic buffer overflows, use-after-free). The correct Rust translation must *not* reproduce the bug. These pairs train the model to infer intent, not transliterate behavior.

6.2 Verification Pipeline for Training Data

Every candidate training pair (C_i, R_i) must pass:

1. R_i compiles with `rustc` (no errors).

2. R_i contains zero `unsafe` blocks (or minimally scoped, justified `unsafe`).

3. R_i passes Miri with no UB detected.

4. Differential fuzzing: $\text{output}(C_i, x) = \text{output}(R_i, x)$ for 10^4 random inputs x.

5. R_i passes Clippy with no warnings (idiomatic Rust check).

6. R_i's cyclomatic complexity is within 2x of a human-written equivalent (readability check).

Pairs that pass all six gates enter the training corpus. Pairs that fail gate 4 (semantic equivalence) are discarded. Pairs that fail gates 5–6 (idiomaticity) are flagged for human review.

7 Unit Economics and Business Model

7.1 Pricing Model

Table 4: Pricing Tiers

Tier	Description	Price
Open Source	Frontend analysis + CAB format + verification harness	Free
Developer	Fine-tuned model API, 10K LOC/month, basic verification	$500/mo
Professional	Claude-augmented translation, 100K LOC/month, full verification	$5,000/mo
Enterprise	Unlimited, dedicated instance, human review platform, SLA	$50K+/mo
Defense	Air-gapped deployment, formal verification, DO-178C/IEC 62304	Custom

7.2 Unit Economics

- **Cost per function translation (fine-tuned model):** $0.001–0.01 (inference cost on hosted GPU)
- **Cost per function translation (Claude):** $0.05–0.50 (API cost, depending on context length)
- **Cost per verified pair (training data):** $0.10–1.00 (Claude generation + verification compute)
- **Target gross margin:** 70–85% at scale (after fine-tuned model handles majority of volume)

The distillation flywheel is the key to unit economics: as the fine-tuned model improves, more functions route to it instead of Claude, reducing marginal cost while maintaining quality.

8 Risk Analysis

Table 5: Risk Matrix

Risk	**Description**	**Likelihood**	**Mitigation**
LLM plateau	Translation quality hits ceiling	Medium	Formal methods compensate
Microsoft competition	Internal tools released externally	High	Speed to market, defense niche
Rust evolution	Borrow checker changes break tools	Low	Track Rust RFC process
DARPA pivots	TRACTOR defunded or redirected	Low	Commercial customers primary
Training data IP	Copyright issues with mined pairs	Medium	Focus on synthetic generation
False equivalence	Verified translations contain subtle bugs	High	Multi-layer verification

9 Technical Foundations: Type Theory of the Translation

9.1 The Translation as Type Enrichment

Let $\mathcal{C}$ denote the type system of C and $\mathcal{R}$ denote the type system of Rust. The C type system is a *subsystem* of Rust's in a precise sense: every C type has a Rust equivalent, but Rust's type system includes additional structure (lifetimes, ownership, trait bounds) that C lacks.

Definition 9.1 (Type Enrichment)**.** A C→Rust translation is a functor $T : \mathcal{C} \to \mathcal{R}$ that maps each C type τ_C to a Rust type $T(\tau_C)$ such that:

1. T preserves the data layout: $\text{sizeof}(\tau_C) = \text{sizeof}(T(\tau_C))$
2. T adds lifetime annotations: if $\tau_C = \text{int*}$, then $T(\tau_C) =$ `&'a i32` for some lifetime `'a`
3. T adds ownership semantics: if τ_C is a heap pointer, $T(\tau_C) \in \{$`Box<T>`, `Rc<T>`, `Arc<T>`, `&T`, `&mut T`$\}$

The hard problem is that T is not unique—there are many valid Rust types for each C pointer. The correct choice depends on usage context, which is exactly the information that a general-purpose LLM lacks but the OSG provides.

9.2 Connection to the Yoneda Perspective

From the Yoneda lemma's perspective, a C pointer is fully characterized by *how it is used*—the set of all operations applied to it across the program. The ownership classification in Stage 2 is precisely the construction of this representable functor: we observe all uses of a pointer (its "Yoneda image") and infer the minimal Rust type that supports exactly those uses.

Theorem 9.1 (Yoneda Ownership Inference)**.** Let p be a pointer in a C program P, and let $U(p) = \{u_1, \ldots, u_n\}$ be the set of all operations applied to p across P. The correct Rust ownership type for p is the **minimal** Rust type τ such that $\mathrm{Hom}_{\mathcal{R}}(\tau, -) \supseteq U(p)$.

This is the Compile-Time Supremacy principle expressed category-theoretically: the entire C→Rust translation reduces to computing the Yoneda embedding of each pointer's usage pattern into Rust's type lattice.

10 Conclusion

The C→Rust migration opportunity is real, immediate, and structurally favored by government mandate, enterprise commitment, and technological readiness. The thesis is simple: *general-purpose LLMs cannot solve this problem alone, but a hybrid system combining LLM reasoning with static analysis, formal verification, and progressive distillation can.*

The Ferrous Bridge architecture—open-source infrastructure creating ecosystem lock-in, proprietary intelligence creating competitive advantage, Claude as the reasoning backbone progressively distilled into specialized models—represents a practical path to capturing this market.

The build path is concrete: 14 months to a production beta, using Claude Code agent teams for development, Claude API for translation, and a formal verification pipeline as the quality gate. The open-source components create community and trust. The proprietary components create defensibility and margin.

The Compile-Time Supremacy thesis provides the theoretical grounding: every line of C translated to safe Rust is an invariant promoted from runtime hope to compile-time proof. The business captures value proportional to the security value created. And the Yoneda perspective reveals why this works at a foundational level: the correct Rust type for any C construct is determined by *how that construct is used*, which is precisely the information that a well-designed analysis pipeline can extract and a well-trained model can interpret.

The market window is open. DARPA is funding. Microsoft is building internally. The tooling gap is acute. The time to build is now.

References

[1] DARPA. "TRACTOR: Translating All C to Rust." https://www.darpa.mil/research/programs/translating-all-c-to-rust, 2024.

[2] ONCD. "Back to the Building Blocks: A Path Toward Secure and Measurable Software." White House Office of the National Cyber Director, 2024.

[3] Chandrasekaran, V. et al. "ForCLift: Formally-Verified Compositional Lifting of C to Rust." DARPA TRACTOR Award, 2025.

[4] Immunant. "C2Rust: C to Rust Translation Framework." https://c2rust.com, 2024.

[5] Hunt, G. "Eliminating C and C++ from Microsoft by 2030." LinkedIn, December 2025.

[6] Long, M. "The Minimal Runtime Axiom: Compile-Time Supremacy as a Foundational Principle." YonedaAI Research Collective, 2025.

[7] Long, M. "ContextFS: A Type Memory Database for AI Agent Systems." YonedaAI Research Collective, 2025.

[8] Matsakis, N. and Klock, F. "The Rust Language." *ACM SIGADA Ada Letters*, 34(3), 2014.

[9] Jung, R. et al. "RustBelt: Securing the Foundations of the Rust Programming Language." *POPL*, 2018.

[10] Amazon Web Services. "Kani Rust Verifier." https://github.com/model-checking/kani, 2023.

[11] Denis, X. et al. "Creusot: A Foundry for the Deductive Verification of Rust Programs." *ICFEM*, 2022.

[12] CISA et al. "The Case for Memory Safe Roadmaps." Joint cybersecurity advisory, 2023.